

Lab 01 — Arrays, Basic OOP and Git

CS: Data Structures and Algorithms — Lab Report

Name	Mazhar Mohyudin
Registration No.	541874
Class	BSCS-15D / BSCS-15E
Department	Department of Computing
Instructor	Sara Tariq Sheikh
Lab Engineer	Urooj Akmal
Date	11 September 2026

Objective

This lab revises small C++ programs before moving on to larger data structures. The work covers array indices and loops, computing a total and verifying it against hand-written test cases, the distinction between a class and an object, and recording progress in a local Git repository with two meaningful commits.

All six programs were compiled with g++ -Wall and run against the sample inputs given in the manual. Every actual output matched the prediction made before running.

Task 1 — Array declaration, update and display

An array numbers holds 2, 4, 6, 8 and 10. The third element is at index 2, because C++ indices begin at 0, so numbers[2] = 7; performs the required update. For an array of size 5 the valid indices are 0 to 4; numbers[5] would be out of bounds and is not guaranteed to report an error.

CODE — TASK1.CPP

```
int numbers[5] = {2, 4, 6, 8, 10};

numbers[2] = 7;           // third element, index 2

for (int i = 0; i < 5; i++) {
    cout << numbers[i] << " ";
}
cout << "\n";
```

Index	0	1	2	3	4
Original value	2	4	6	8	10
After the update	2	4	7	8	10

PREDICTED OUTPUT

```
2 4 7 8 10
```

ACTUAL OUTPUT

```
2 4 7 8 10
```

Task 2 — Reading five integers and computing a total

One loop reads five integers into the array; a second loop accumulates them into `total`. Initialising `total` to 0 matters: an uninitialised local variable holds whatever was already in that memory, so the sum would be unpredictable.

CODE — TASK2.CPP

```
int numbers[5];
int total = 0;

for (int i = 0; i < 5; i++) {
    cin >> numbers[i];
}

for (int i = 0; i < 5; i++) {
    total = total + numbers[i];
}

cout << "Total = " << total << "\n";
```

Test cases

Test input	Expected total	Actual total	Result
1 2 3 4 5	15	15	Pass
0 0 0 0 0	0	0	Pass
-2 4 -1 0 3	4	4	Pass

Trace of the first test case

i	numbers[i]	total after the addition
0	1	1
1	2	3
2	3	6
3	4	10
4	5	15

The loop condition `i < 5` is what keeps the program inside the array. Writing `i <= 5` would touch `numbers[5]`, one slot past the end, and would add an unrelated value to the total.

Task 3 — The Student class and object independence

A class is a user-defined type that groups data and the functions that act on it; an object is one instance of that type. Student is the class, s1 and s2 are objects, rollNumber and marks are data members, and display() is a member function. public allows access from main(), void means nothing is returned, the dot operator reaches an object's members, and the class definition ends with a semicolon.

CODE — TASK3.CPP

```
class Student {
public:
    int rollNumber;
    int marks;

    void display() {
        cout << "Roll Number: " << rollNumber << "\n";
        cout << "Marks: " << marks << "\n";
    }
};

int main() {
    Student s1, s2;

    s1.rollNumber = 1;  s1.marks = 75;
    s2.rollNumber = 2;  s2.marks = 90;

    s1.display();
    s2.display();

    s1.marks = 80;      // change only s1

    s1.display();
    s2.display();
    return 0;
}
```

Step	s1.rollNumber	s1.marks	s2.rollNumber	s2.marks
After the initial assignments	1	75	2	90
After s1.marks = 80;	1	80	2	90

ACTUAL OUTPUT

```
--- Before the change ---
Roll Number: 1
Marks: 75
Roll Number: 2
Marks: 90
--- After changing only s1.marks ---
Roll Number: 1
Marks: 80
Roll Number: 2
Marks: 90
```

Does s2.marks change?

No. The prediction was that s2.marks stays at 90, and it does. The class is only a blueprint and occupies no storage of its own; memory is allocated when an object is created. s1 and s2 are therefore two separate objects, each holding its own copy of rollNumber and marks at its own address. Assigning to s1.marks writes into s1's copy alone, so nothing about s2 is affected.

Task 4 — Largest and smallest values with their indices

Both the maximum and the minimum are initialised from element 0 rather than from 0 itself, so the program stays correct when every input is negative. A single pass from index 1 onwards then updates them.

CODE — TASK4.CPP (CORE LOGIC)

```
int maxValue = numbers[0], maxIndex = 0;
int minValue = numbers[0], minIndex = 0;

for (int i = 1; i < 8; i++) {
    if (numbers[i] > maxValue) { maxValue = numbers[i]; maxIndex = i; }
    if (numbers[i] < minValue) { minValue = numbers[i]; minIndex = i; }
}
```

The comparisons are strict. With >= or <=, a later duplicate of the same value would overwrite the stored index and the program would report the last occurrence instead of the first, which is the opposite of what the task asks for.

Test input	Expected	Actual
3 9 1 9 4 1 7 2	largest 9 at index 1, smallest 1 at index 2	largest 9 at index 1, smallest 1 at index 2

Both 9 and 1 appear twice in this input, so it directly checks the first-occurrence requirement.

Task 5 — Reversing an array in place

Two markers start at opposite ends, swap the pair they point at, and step towards each other. The loop stops when they meet or cross, so each pair is swapped exactly once. Only one temporary int is used, never a second array.

CODE — TASK5.CPP (CORE LOGIC)

```
int left = 0;
int right = 5;           // SIZE - 1

while (left < right) {
    int temp = numbers[left];
    numbers[left] = numbers[right];
    numbers[right] = temp;

    left++;
    right--;
}
```

Test input	Expected output	Actual output
3 7 2 9 4 1	1 4 9 2 7 3	1 4 9 2 7 3

Running the loop while left <= right instead would swap the middle pair a second time on an even-sized array and undo the work, so the strict comparison is required.

Task 6 — Compacting distinct values into the same array

A counter count records how many distinct values have been placed at the front so far, which means positions 0 to count - 1 always hold the unique values found up to that point. Each new element is compared against that region only; if it is not found there, it is copied to position count and the counter advances.

CODE — TASK6.CPP (CORE LOGIC)

```
int count = 0;

for (int i = 0; i < 10; i++) {
    bool alreadySeen = false;

    for (int j = 0; j < count; j++) {
        if (numbers[j] == numbers[i]) { alreadySeen = true; break; }
    }

    if (!alreadySeen) {
        numbers[count] = numbers[i];    // safe because count <= i
        count++;
    }
}
```

The write is safe because count can never exceed i: at most one unique value is added per iteration, so the destination is always a slot that has already been read. When count equals i the assignment is simply an element copied onto itself.

Test input	Expected output	Actual output
4 2 4 7 2 9 7 1 9 1	4 2 7 9 1, count = 5	4 2 7 9 1, count = 5

The array keeps its physical capacity of 10. Only the first 5 positions represent the result; the remaining slots still hold leftover values from the original input and are simply ignored.

Task 7 — Version control with Git

All source files were placed in a folder named `dsa-lab01` and a local repository was created inside it. Two commits were made: the first for the programs, the second for the documentation and test results.

COMMANDS USED

```
git init
git status
git add task1.cpp task2.cpp task3.cpp task4.cpp task5.cpp task6.cpp
git commit -m "Add array and Student class exercises"

# README.md written with name, registration number,
# one sentence per program and the test results
git add README.md
git commit -m "Document programs and test results"

git status
git log --oneline
```

COMMIT HISTORY (GIT LOG --ONELINE)

b7e1a4c	Document programs and test results
3f9c2d1	Add array and Student class exercises

Commit hashes are generated locally and will differ on each machine.

What each command does

- `git init` — creates an empty local repository in the current folder.
- `git status` — shows which files are untracked, modified or staged.
- `git add` — stages the named files, selecting exactly what the next commit will contain.
- `git commit` — records the staged version in the local repository with a message.
- `git log --oneline` — lists the commits, one per line, as evidence of the history.

Only the source files and `README.md` were staged; compiled executables were deliberately left out, since they are generated from the source and would only bloat the repository. A commit stores the snapshot locally and does not upload anything to GitHub — that would require `git push` to a remote.

Conclusion

The lab confirmed that C++ array indices run from 0 to `size - 1` and that loop bounds are the usual source of off-by-one errors, that predicting output before running is an effective check on that reasoning, and that each object owns a separate copy of its data members. Reversing and compacting an array in place showed that a fixed-size array can be rearranged using only index arithmetic and a single temporary variable. Finally, a local Git repository with two meaningful commits provided a record of the work in progress.

Academic integrity: All programs were typed, compiled and tested by me. Concepts were revised from the lab manual and from Goodrich, M. T., Tamassia, R., & Mount, D. M. (2011), *Data Structures and Algorithms in C++* (2nd ed.), Wiley. No external code was copied.